College of Engineering, Construction and Living Sciences
Bachelor of Information Technology
ID608001: Intermediate Application Development Concepts
Level 6, Credits 15

Project

Assessment Overview

In this **individual** assessment, you will develop **four** frontend applications. In addition, marks will be allocated for code quality and best practices, documentation and **Git** usage.

Learning Outcomes

At the successful completion of this course, learners will be able to:

1. Apply design patterns and programming principles using software development best practices.
2. Design and implement full-stack applications using industry relevant programming languages.

Assessments

Assessment	Weighting	Due Date	Learning Outcome
Practical	20%	Thursday 15th May at 2.59 PM	1
Project	80%	Monday 9th June at 7.59 AM	1

Conditions of Assessment

You will complete this assessment during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by **Monday, 9 June 2025** at **7.59 AM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of **50%** over all assessments in **ID608001: Intermediate Application Development Concepts**.

Authenticity

All parts of your submitted assessment **must** be completely your work. Do your best to complete this assessment without using an **AI generative tool**. You need to demonstrate to the course lecturer that you can meet the learning outcome for this assessment.

Learning to use AI tool is an important skill. While AI tools are powerful, you **must** be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You **must** still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository **README.md** file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

It also applies to code snippets retrieved from **StackOverflow** and **GitHub**.

Failure to do this may result in a mark of **zero** for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with **Otago Polytechnic** policies. Learners can view policies on the **Otago Polytechnic** website located at https://www.op.ac.nz/about-us/governance-and-management/policies.

Submission

You **must** submit all application files via **GitHub**. Create a repository and add the course lecturer as a collaborator. Late submissions will incur a **10% penalty per day**, rolling over at **8:00 AM**.

Extensions

Familiarise yourself with the assessment due date. Extensions will **only** be granted if you are unable to complete the assessment by the due date because of **unforeseen circumstances outside your control**. The length of the extension granted will depend on the circumstances and **must** be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments **are not** applicable in **ID608001: Intermediate Application Development Concepts**.

Instructions

Functionality - Learning Outcomes 1 and 2 (50%)

Milestone One - Due Monday, 7 April 2025 at 7.59 AM

- **Movie Listings Application (10%):**
  - Create a new directory called **movie-listings-application** and create a new **React** application using **Vite**.
  - Deploy the application to **Vercel**.
  - Use **React Query/TanStack Query** and the **Fetch API** to fetch data from the **The Movie DB API**.
  - Navigation bar using **Tailwind CSS**, **Shadcn UI** and **React Router** with the following movie type options:
    - Trending. This is the default option.
    - Top Rated
    - Action
    - Animation
    - Comedy
  - The endpoints for the movie types are as follows:
    - Trending - https://api.themoviedb.org/3/trending/all/week?api_key=<API KEY>&language=en-US
    - Top Rated - https://api.themoviedb.org/3/movie/top_rated?api_key=<API KEY>&language=en-US
    - Action - https://api.themoviedb.org/3/discover/movie?api_key=<API KEY>&with_genres=28
    - Animation - https://api.themoviedb.org/3/discover/movie?api_key=<API KEY>&with_genres=16
    - Comedy - https://api.themoviedb.org/3/discover/movie?api_key=<API KEY>&with_genres=35

    Note: Replace <API KEY> with a **The Movie DB API** key given by your course lecturer.
  - When an option is selected, display the title, overview, poster path and release date of the first 10 movies.
    - Style each movie to look like a card using **Tailwind CSS** and **Shadcn UI**.
    - Display five movies per row.
  - Handle any missing data or errors gracefully.

Milestone Two - Monday, 5 May 2025 at 7.59 AM

- **Hacker News Application (20%):**
  - Create a new directory called **hacker-news-application** and create a new **React** application using **Vite**.
  - Deploy the application to **Vercel**.
  - Use **React Query/TanStack Query** and **Fetch API** to fetch data from the **Hacker News API**.
  - Navigation bar using **Tailwind CSS**, **Shadcn UI** and **React Router** with the following story options:
    - Ask Stories. This is the default option.
    - Best Stories
    - Job Stories
    - New Stories
    - Show Stories
    - Top Stories
    - Leaders
  - The endpoints for the stories are as follows:
    - Ask Stories - https://hacker-news.firebaseio.com/v0/askstories.json?print=pretty
    - Best Stories - https://hacker-news.firebaseio.com/v0/beststories.json?print=pretty
    - Job Stories - https://hacker-news.firebaseio.com/v0/jobstories.json?print=pretty
    - New Stories - https://hacker-news.firebaseio.com/v0/newstories.json?print=pretty
    - Show Stories - https://hacker-news.firebaseio.com/v0/showstories.json?print=pretty
    - Top Stories - https://hacker-news.firebaseio.com/v0/topstories.json?print=pretty

- When an option is selected, display the title of the first 25 stories.
  - Style each story to look like a card using **Tailwind CSS** and **Shadcn UI**.
  - Display five stories per row.
- When a story is clicked, the user will be navigate to a page that displays the following information:
  - By
  - Kids. **Note:** This is an array of ids. If the array is empty, display **N/A**. Display the first five ids as URLs in this format: https://hacker-news.firebaseio.com/v0/item/<Id>.json?print=pretty. When clicked, these URLs will open in a new tab.
  - Score
  - Time. **Note:** Convert the time to a readable format.
  - Title
  - Type
  - URL. **Note:** When clicked, this URL will open in a new tab.

  This information is fetched from the following endpoint: https://hacker-news.firebaseio.com/v0/item/<Id>.json?print=pretty.
- On a page, create a form that allows the user to search a leader by their name. You can get the leader's name from the following URL: https://news.ycombinator.com/leaders. **Note:** The **Hacker News API** does not provide leader information. You will need to hardcode the leader information in your application. When the form is submitted, display the leader's information such as:
  - About
  - Created. **Note:** Convert the time to a readable format.
  - Id
  - Karma
  - Submitted. **Note:** This is an array of ids. Display the first 5 ids as URLs in this format: https://hacker-news.firebaseio.com/v0/item/<Id>.json?print=pretty.

  This information is fetched from the following endpoint: https://hacker-news.firebaseio.com/v0/user/<Id>.json?print=pretty.
- Style the leader information to look like a card using **Tailwind CSS** and **Shadcn UI**.
- Handle any missing data or errors gracefully.

Milestone Three - Due Monday, 26 May 2025 at 7.59 AM

- **Trivia Application (25%):**
  - Create a new directory called **trivia-application** and create a new **React** application using **Vite**.
  - Use **React Query/TanStack Query** and **Fetch API** to fetch data from the **OpenTDB API**.
  - Create a form with the following inputs:
    - Name of the user. This is a text input field. The default value is **Anonymous**.
    - Amount. This is the number of questions to retrieve. The default value is 10.
    - Category. This is a dropdown list. The options can be retrieved from the following endpoint: https://opentdb.com/api_category.php. The default option is **Any Category**.
    - Difficulty. This is a dropdown list with the following options:
      - Any Difficulty. This is the default option.
      - Easy.
      - Medium
      - Hard
    - Type. This is a dropdown list with the following options:
      - Any Type. This is the default option.
      - Multiple Choice
      - True/False
  - When the form is submitted, perform a request to the https://opentdb.com/api_config.php.

  - Use the response to display the questions and answers in a quiz format.
  - Create a form that allows the user to submit their answers. After the user submits their answers, display their name, score and the correct answers.
  - Store the user's name, score and category in an **array** of **objects** in **localStorage**.
  - Display a leaderboard that shows the top 5 scores for each category stored in **localStorage**.
  - Style the application using **Tailwind CSS** and **Shadcn UI**.
  - Handle any missing data or errors gracefully.

Milestone Four - Due Monday, 9 June 2025 at 7.59 AM

- **Emerging Technologies Application (10%):**
  - Create a new directory called **emerging-technologies-application** and create a new **Astro** or **Solid.js** application.
  - The user should be able to:
    * Add a new todo.
    * Delete a todo.
    * Edit a todo.
    * Mark a todo as complete.
    * Mark a todo as incomplete.
    * View all todos.
    * View completed todos.
    * View incomplete todos.
  - Style the application using **Tailwind CSS**.
  - Handle any missing data or errors gracefully.

Code Quality and Best Practices - Learning Outcome 1 (40%)

- Project Structure - The codebase should be well-organised with a clear and logical structure that separates concerns and promotes reusability and maintainability.
- Naming Conventions - File, component, function and variable names should be clear, descriptive and consistent across the codebase.
- Documentation and Comments - The codebase should be well-documented using the **JSDoc** format with comments that explain complex logic and clarify the purpose of classes, functions or complex sections.
- Code Formatting - Consistent indentation and spacing should be used across the codebase. It includes ensuring proper indentation for code blocks, following a standard format for braces and adding blank lines between sections. It should be automated using a tool like **Prettier**.
- Dead Code - The codebase should not contain unused or redundant files, components, functions or variables. Keeping unnecessary codebase around can lead to confusion, clutter and potential bugs down the line.
- Performance and Scalability - The codebase should be optimised for performance, using efficient algorithms and data structures to minimise bottlenecks.

Documentation and Git Usage - Learning Outcome 1 (5%)

- **GitHub** issues to help you organise and prioritise your development work. The course lecturer needs to see consistent use of **GitHub** issues for the duration of the assessment.
- Provide URLs to the applications on **Vercel** in your repository **README.md** file
- Use of **Markdown**, i.e., headings, bold text, code blocks, etc.
- Correct spelling and grammar.

- A **.gitignore** file containing the ignored files in this resource - https://raw.githubusercontent.com/github/gitignore/main/Node.gitignore.
- Your **Git commit messages** should:
  - Reflect the context of each functional requirement change.
  - Be formatted using an appropriate naming convention style.

Additional Information

- **Do not** rewrite your **Git** history. It is important that the course lecturer can see how you worked on your assessment over time.
- You need to show the course lecturer the initial **GitHub** issues before you start your development work. Following this, you need to show the course lecturer your **GitHub** issues at the end of each week.